



Patrones de Diseño - Sanos y Salvos API

Resumen General

Este proyecto implementa **4 patrones de diseño** en el backend para garantizar un código limpio, mantenible y escalable. Cada patrón fue seleccionado para resolver un problema concreto de la arquitectura.

#	Patrón	Ubicación	Problema que Resuelve
1	Repository	src/repositories/	Desacoplamiento del acceso a datos
2	Strategy	src/strategies/	Algoritmos de notificación intercambiables
3	Builder	src/builders/	Construcción de objetos complejos (Reportes)
4	Factory	src/factories/	Creación centralizada de servicios

Además, se implementa un patrón arquitectónico **BFF (Backend For Frontend)** donde Express actúa como capa intermedia entre el frontend y Supabase.

1. Repository Pattern

Archivos

- `src/repositories/BaseRepository.ts` — Clase abstracta genérica
- `src/repositories/MascotaRepository.ts`
- `src/repositories/UsuarioRepository.ts`
- `src/repositories/ReporteRepository.ts`
- `src/repositories/MensajeRepository.ts`
- `src/repositories/SoporteTicketRepository.ts`

¿Qué problema resuelve?

En el proyecto original Java, los controladores accedían directamente a los repositorios JPA sin capa de abstracción. Esto acoplaba la lógica de negocio al ORM específico, haciendo imposible cambiar de base de datos o hacer testing sin una base de datos real.

¿Por qué se eligió este patrón?

- **Desacopla** la lógica de negocio del acceso a datos (Supabase)

- Permite **cambiar de proveedor** de datos sin modificar los servicios
- Facilita enormemente el **testing** con repositorios mock
- Proporciona una **interfaz uniforme** (CRUD) para todas las entidades

¿Cómo mejora la mantenibilidad?

```
// Interfaz genérica que define el contrato
export interface IRepository<T> {
  findAll(): Promise<T[]>;
  findById(id: number): Promise<T | null>;
  create(entity: Partial<T>): Promise<T>;
  update(id: number, entity: Partial<T>): Promise<T | null>;
  delete(id: number): Promise<boolean>;
}

// Cada repositorio extiende la base y agrega métodos específicos
export class MascotaRepository extends BaseRepository<Mascota> {
  async findByEstado(estado: string): Promise<Mascota[]> { ... }
}
```

Los servicios reciben la interfaz, no la implementación concreta:

```
export class MascotaService {
  constructor(private repository: IMascotaRepository) {}
  // El servicio NO sabe si es Supabase, PostgreSQL directo, o un mock
}
```

2. Strategy Pattern

Archivos

- `src/strategies/NotificationStrategy.ts`

¿Qué problema resuelve?

El sistema necesita enviar notificaciones por diferentes canales (email, SMS, push) cuando se crea un reporte o se encuentra una mascota. Sin el patrón Strategy, se usarían condicionales `if/else` para cada tipo, violando el principio Open/Closed.

¿Por qué se eligió este patrón?

- Permite **agregar nuevos canales** sin modificar código existente
- Cada estrategia se puede **testear de forma aislada**
- El canal se puede **cambiar en tiempo de ejecución**
- Elimina cadenas de `if/else` o `switch`

¿Cómo mejora la mantenibilidad?

```
// Interfaz Strategy
interface INotificationStrategy {
  readonly channel: string;
  send(data: NotificationData): Promise<NotificationResult>;
}

// Estrategias concretas
class EmailNotificationStrategy implements INotificationStrategy { ... }
class SmsNotificationStrategy implements INotificationStrategy { ... }
class PushNotificationStrategy implements INotificationStrategy { ... }

// Contexto: usa la estrategia seleccionada
class NotificationService {
  setStrategy(strategy: INotificationStrategy): void { ... }
  async notify(data: NotificationData): Promise<NotificationResult> { ... }
}
```

Para agregar un nuevo canal (ej: WhatsApp), solo se crea una nueva clase que implemente `INotificationStrategy`. No se modifica ningún código existente.

3. Builder Pattern

Archivos

- `src/builders/ReporteBuilder.ts`

¿Qué problema resuelve?

Los reportes son objetos complejos con múltiples campos opcionales y reglas de validación. En el proyecto original, se creaban directamente en el controlador sin validación, lo que provocaba datos inconsistentes en la base de datos.

¿Por qué se eligió este patrón?

- **Validación centralizada** antes de crear el objeto
- API **fluida** que hace el código más legible
- Separa la **lógica de construcción** del controlador
- El `ReporteDirector` proporciona **plantillas predefinidas**

¿Cómo mejora la mantenibilidad?

```
// Construcción fluida con validación
const reporte = new ReporteBuilder()
  .setDescripcion('Se perdió un perro labrador')
  .setUbicacion('Av. Providencia 1234')
  .setTipoReporte('perdido')
  .setMascotaId(1)
  .setUsuarioId(1)
  .build(); // ← Valida antes de crear

// Director para casos comunes
const reporteEmergencia = ReporteDirector.crearReporteEmergencia(
  'Animal herido',
  'Maipú',
  3
);
```

El método `build()` valida:

- Descripción obligatoria y no vacía
- Tipo de reporte válido (`perdido` , `encontrado` , `avistamiento` , `maltrato` , `emergencia`)
- Ubicación obligatoria

4. Factory Pattern

Archivos

- `src/factories/ServiceFactory.ts`

¿Qué problema resuelve?

La creación de servicios requiere inyectar repositorios como dependencias. Sin un Factory, cada controlador necesitaría conocer y crear todas las dependencias, acoplando el código y duplicando la lógica de instanciación.

¿Por qué se eligió este patrón?

- **Centraliza** la creación de todas las dependencias
- Implementa **Dependency Inversion** (los controladores no crean sus dependencias)
- Facilita **cambiar implementaciones** (ej: para testing)
- **Single Responsibility**: la configuración de dependencias está en un solo lugar

¿Cómo mejora la mantenibilidad?

```
class ServiceFactory {
  // Crear servicios con sus dependencias inyectadas
  static createMascotaService(repo?: IMascotaRepository): MascotaService {
    return new MascotaService(repo || ServiceFactory.createMascotaRepository());
  }

  // Permite inyectar mocks para testing
  static createAuthService(repo?: IUserRepository): AuthService {
    return new AuthService(repo || ServiceFactory.createUsuarioRepository());
  }

  // Crea servicios de notificación con la estrategia apropiada
  static createNotificationService(channel: string = 'email'): NotificationService {
    .. }
  }
}
```

En el servidor:

```
// server.ts – creación limpia de servicios
const authService = ServiceFactory.createAuthService();
const mascotaService = ServiceFactory.createMascotaService();
```



Patrón Arquitectónico: BFF (Backend For Frontend)

¿Qué es?

La API Express actúa como **Backend For Frontend**, una capa intermedia entre el frontend React y Supabase que:

- Implementa lógica de negocio y validación
- Maneja autenticación JWT
- Transforma datos para el frontend
- Oculta la complejidad de Supabase al frontend

Beneficios

- El frontend solo conoce la API REST, no Supabase directamente
- Permite cambiar la base de datos sin afectar el frontend
- Centraliza la seguridad y validación en el backend
- Desplegable como serverless en Vercel



Tabla Resumen de Principios SOLID Aplicados

Principio	Patrón que lo implementa	Ejemplo
S ingle Responsibility	Factory, Repository	Cada clase tiene una sola responsabilidad
O pen/Closed	Strategy	Agregar canales sin modificar código
L iskov Substitution	Repository	Cualquier IRepository funciona igual
I nterface Segregation	Repository, Strategy	Interfaces específicas por dominio
D ependency Inversion	Factory, Repository	Servicios dependen de interfaces, no de implementaciones